CSE344
Software Engineering (SWE)

Week – 3
Requirements
&
Req. Engineering

Requirement (İster) ...

- ***...is a way of expressing a client's need*** (a service or a constraint on the system) ranging at one end from
  - *a high-level abstract statement* at the other end to
  - *a detailed mathematical functional specification*.
- ***...may serve a dual function***
  - *Basis for a bid (öneri) for a contract* ⇔ ***open to interpretation***
  - *Basis for the contract itself* ⇔ ***defined in detail*;**

A Classification of req.s by their reader

- ***User requirements*** (written ***for customers***)
  - *Statements in natural language plus diagrams of the services of the system and its operational constraints.*
- ***System requirements***
  - a *structured* document
  - detailed descriptions of the system's functions, and operational constraints (functionality of the system).
  - *may be part of a contract between client and contractor.*

Definitions and specifications

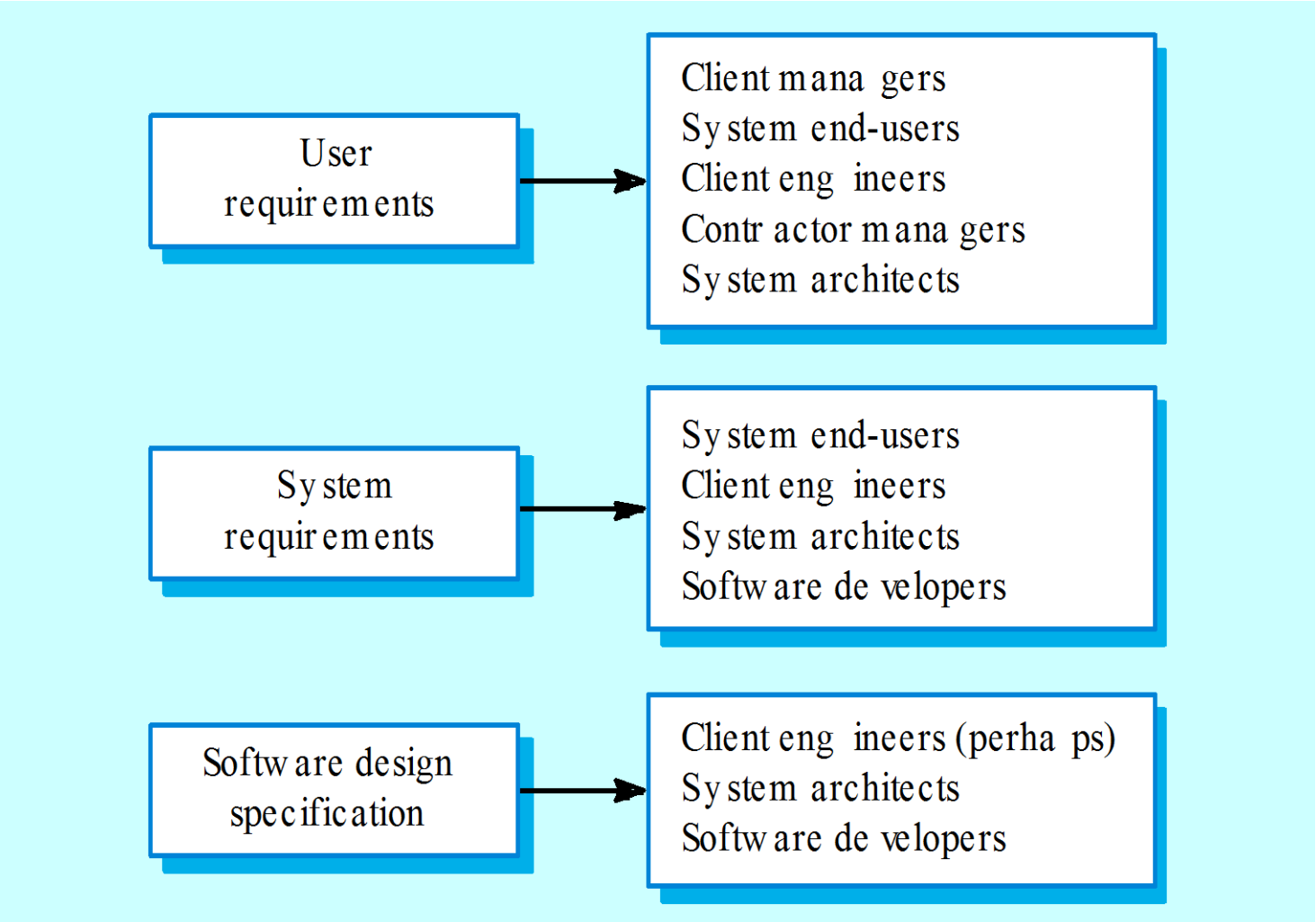
User requirement definition

1. The software must provide a means of representing and accessing external files created by other tools.

System requirements specification

1.1 The user should be provided with facilities to define the type of external files.
1.2 Each external file type may have an associated tool which may be applied to the file.
1.3 Each external file type may be represented as a specific icon on the user's display.
1.4 Facilities should be provided for the icon representing an external file type to be defined by the user.
1.5 When a user selects an icon representing an external file, the effect of that selection is to apply the tool associated with the type of the external file to the file represented by the selected icon.

Requirements readers



Another classification regarding what they state...

- ***Functional requirements***
  - *Statements of services* the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
- ***Non-functional requirements***
  - *constraints on the services or functions* offered by the system such as timing constraints, constraints on the development process, standards, etc.
- ***Domain requirements***
  - *Requirements that come from the application domain* of the system and that *reflect characteristics of that domain*.

The LIBSYS system

- A library system that provides a single interface to a number of databases of articles in different libraries.
- Users can search for, download and print these articles for personal study.

Examples of functional requirements

1. The user shall be able to search either all of the initial set of databases or select a subset from it.
2. The system shall provide *appropriate viewers* for the user to read documents in the document store.
3. Every order shall be allocated a unique identifier (*ORDER_ID*) which the user shall be able to copy to the account's permanent storage area.

Requirements imprecision

- Problems arise when requirements are not precisely stated.
- Ambiguous requirements may be interpreted in different ways by developers and users.
- Consider the term '*appropriate viewers*'
  - User intention - special purpose viewer for each different document type;
  - Developer interpretation - Provide a text viewer that shows the contents of the document.

Requirements completeness and consistency

- In principle, set of requirements should be both complete and consistent.
- ***Completeness***
  - ...includes descriptions of all facilities required.
- ***Consistency***
  - ... no conflicts or contradictions.
- In practice, it is *extremely hard* to produce a complete and consistent requirements document.

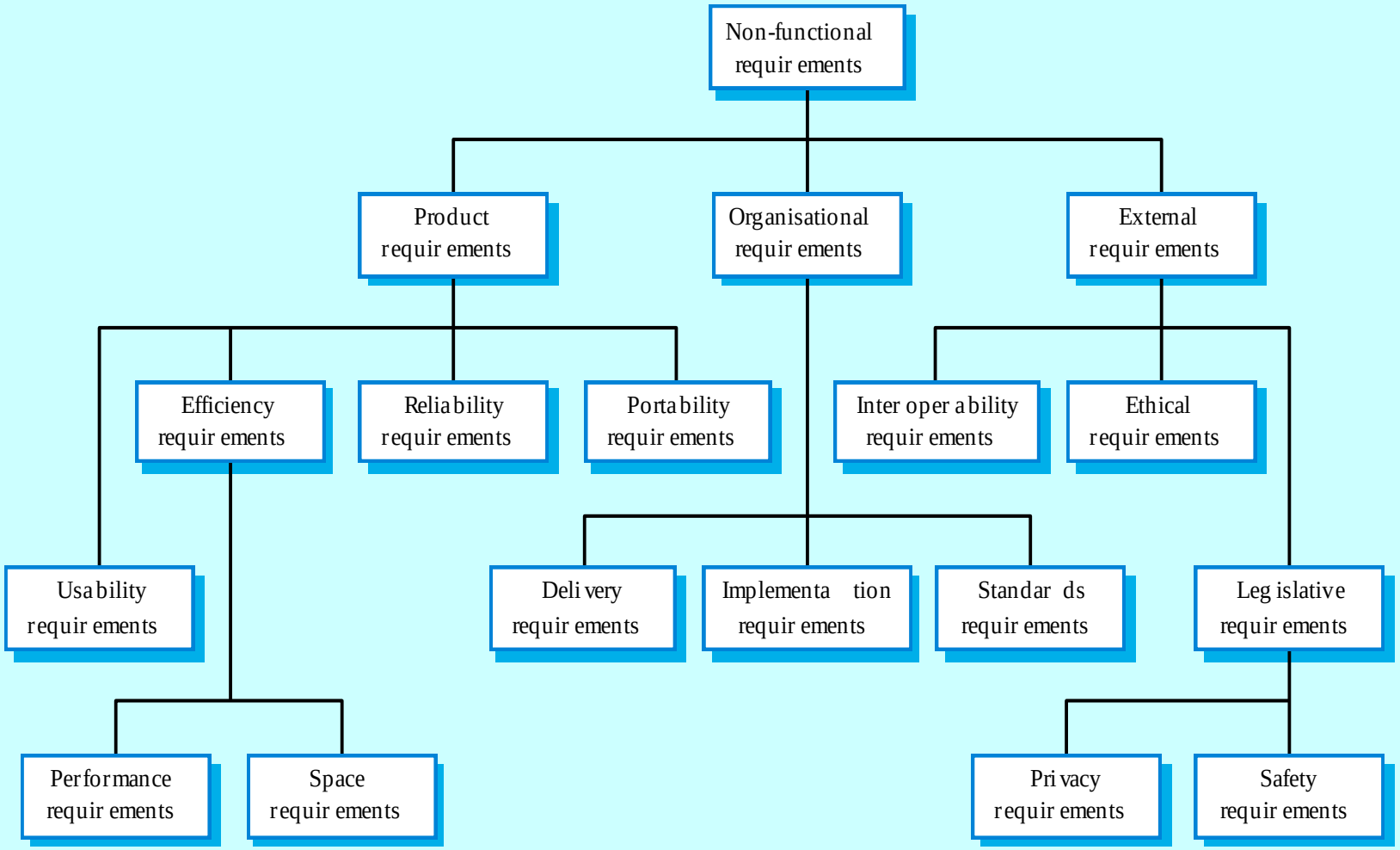
Non-functional (NF) Requirements

- ...define
  - *system properties* such as
    - reliability, response time and storage requirements, etc., and
  - *constraints* such as
    - I/O device capability, system representations, etc.
- *Process requirements* may also be specified *mandating a particular CASE system, programming language or development method*.
- ...are more critical than functional requirements. If these are not met, the system is useless.

Classifications of NF Requirements

- ***Product requirements***
  - ...specify that the delivered product must behave in a particular way e.g. execution speed, reliability, etc.
- ***Organisational requirements***
  - ...are a consequence of organisational policies and procedures e.g. process standards used, implementation requirements, etc.
- ***External requirements***
  - ...arise from factors which are external to the system and its development process e.g. interoperability requirements, legislative requirements, etc.

Types of NF Requirements



Examples of NF Requirements

- ***Product requirement***

  8.1 The user interface for LIBSYS shall be implemented as simple *HTML without frames or Java applets*.

- ***Organisational requirement***

  9.3.2 The system development process and deliverable documents shall conform to the process and deliverables defined in *XYZCo-SP-STAN-95*.

- ***External requirement (legislative here)***

  7.6.5 The system shall not disclose any personal information about customers apart from their name and reference number to the operators of the system.

Verifiability of NF Requirements

- NF requirements ...
  - ...are usually hard to state precisely and hence to verify.
  - ...originate from *user's intention (i.e., a goal)* to state a property about the system
  - ...are *system properties* related to the entirety of the system.
- A ***verifiable*** NF requirement ...
  - ...is a statement using some measure that can be objectively tested.

Example...

- ***A system goal***
  - The system should be easy to use by experienced controllers and should be organised in such a way that user errors are minimised.
- ***A verifiable non-functional requirement***
  - Experienced controllers shall be able to use all the system functions ***after a total of two hours training***. After this training, the ***average number of errors*** made by experienced users shall ***not exceed two per day***.

NF Requirements measures

Property	Measure
Speed	Processed transactions/second User/Event response time Screen refresh time
Size	Mega Bytes Number of ROM chips
Ease of use	Training time Number of help frames
Reliability	Mean time to failure Probability of unavailability Rate of failure occurrence Availability
Robustness	Time to restart after failure Percentage of events causing failure Probability of data corruption on failure
Portability	Percentage of target dependent statements Number of target systems

Requirements interaction

- ***Conflicts between different NF requirements*** are possible in complex systems.
- Example: *Spacecraft system*
  - To minimise weight: use a minimum number of separate chips in the system.
  - To minimise power consumption: use lower-power chips meaning more chips compared with when useing higher-power chips.
  - **Conflict(!!!)**
  - **Question**: *Which is the most critical requirement?*

Domain requirements

- ...originate from the *application domain* and
- ...describe *system characteristics* that reflect the domain.
- ...may be
  - ...new functional requirements, or
  - ...constraints on existing requirements or
  - ...defining specific computations.

Library system domain requirements (Examples 1&2)

1) There shall be a standard user interface to all databases which shall be based on the *Z39.50 standard*.
2) Because of *copyright restrictions*, some documents must be deleted immediately on arrival. Depending on the user's requirements, these documents will either be printed locally on the system server for manually forwarding to the user or routed to a network printer.

Train protection system (Example 3)

3) The deceleration of the train shall be computed as:

   - $D_{train} = D_{control} + D_{gradient}$

   where $D_{gradient}$ is $9.81ms^2$ * compensated gradient/alpha and where the values of $9.81ms^2$ /alpha are known for *different types of train*.

Domain requirements problems

- *Understandability*
  - Requirements are expressed in the language of the application domain;
  - This is often not understood by software engineers developing the system.
- *Implicitness*
  - Domain specialists understand the area so well that they do not think of making the domain requirements explicit.

User requirements

- ... should describe functional and non-functional requirements in such a way that they are *understandable by system users who don't have detailed technical knowledge*.
- ... are *defined using natural language, tables and diagrams* as these can be *understood by all users*.

Problems with natural language

- *Lack of clarity*
  - Precision is difficult without making the document difficult to read.
- *Requirements confusion*
  - Functional and non-functional requirements tend to be mixed-up.
- *Requirements amalgamation*
  - Several different requirements may be expressed together.

System requirements

- More detailed specifications of system functions, services and constraints than user requirements.
- They are intended to be a basis for designing the system.
- They may be incorporated into the system contract.
- System requirements may be defined or illustrated using system models.

Requirements and design

- In principle, *requirements* should state ***what*** the system should do and the *design* should describe ***how*** it does this.
- In practice, requirements and design are inseparable
  - A system architecture may be designed to structure the requirements;
  - The system may inter-operate with other systems that generate design requirements;
  - The use of a specific design may be a domain requirement.

Problems with NL specification

- *Ambiguity*
  - The readers and writers of the requirement must interpret the same words in the same way. NL is naturally ambiguous so this is very difficult.
- *Over-flexibility*
  - The same thing may be said in a number of different ways in the specification.
- *Lack of modularisation*
  - NL structures are inadequate to structure system requirements.

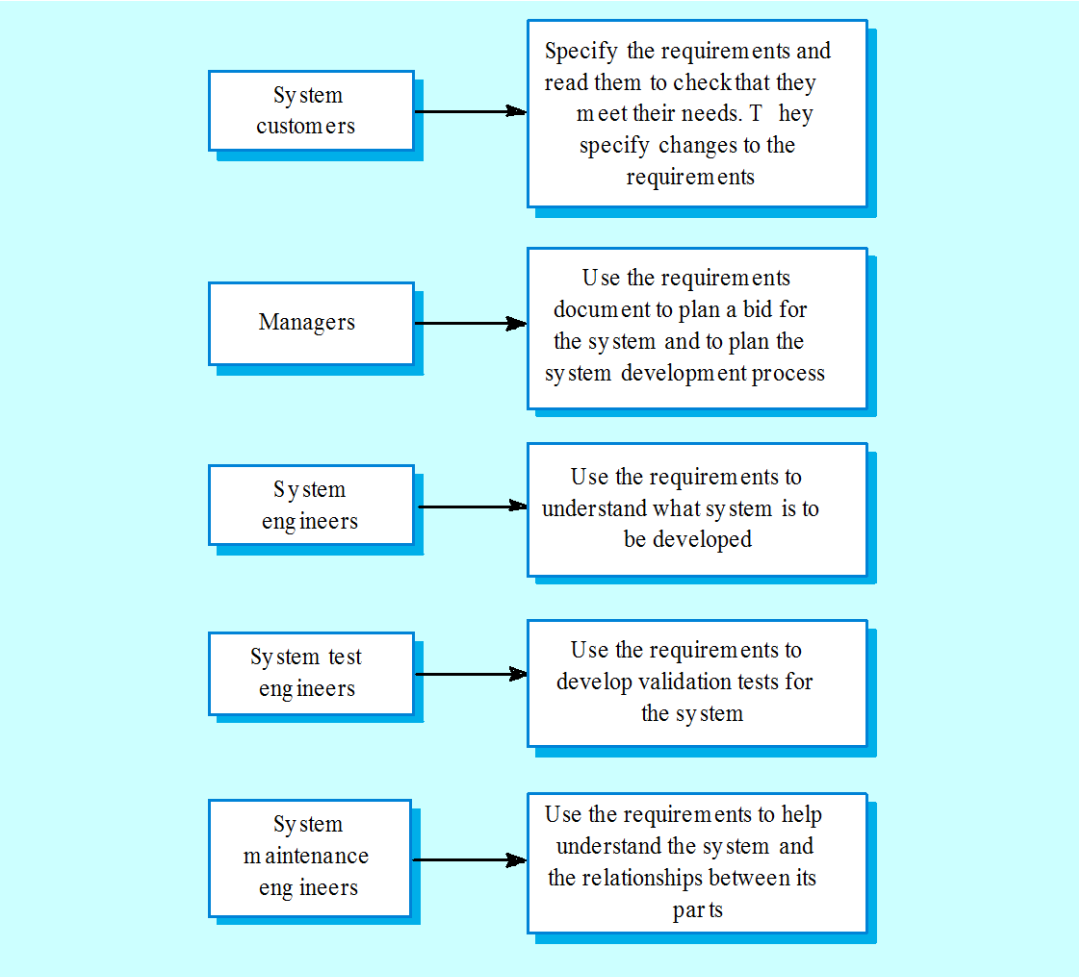
Alternatives to NL specification

- *Structured NL*
  - Form-based or tabular specifications
- *Design description languages (DDLs)*
  - ...use a language like a prog. lang. But with more abstract features to specify req.s by defining an operational model of the system.
  - Not very popular
- *Graphical Notations*
  - UML diagrams such as use-case or sequence diagrams
  - Very popular
- *Mathematical Specifications*
  - State machines or sets
  - Unambiguous, but formal and hard to understand by most customers

The requirements document

- The requirements document is the *official statement of what is required* of the system developers.
- ... includes
  - a definition of *user requirements* and
  - a specification of the *system requirements*.
- NOT a design document.
- It sets WHAT the system should do *rather than* HOW it should do it

Users of a requirements document



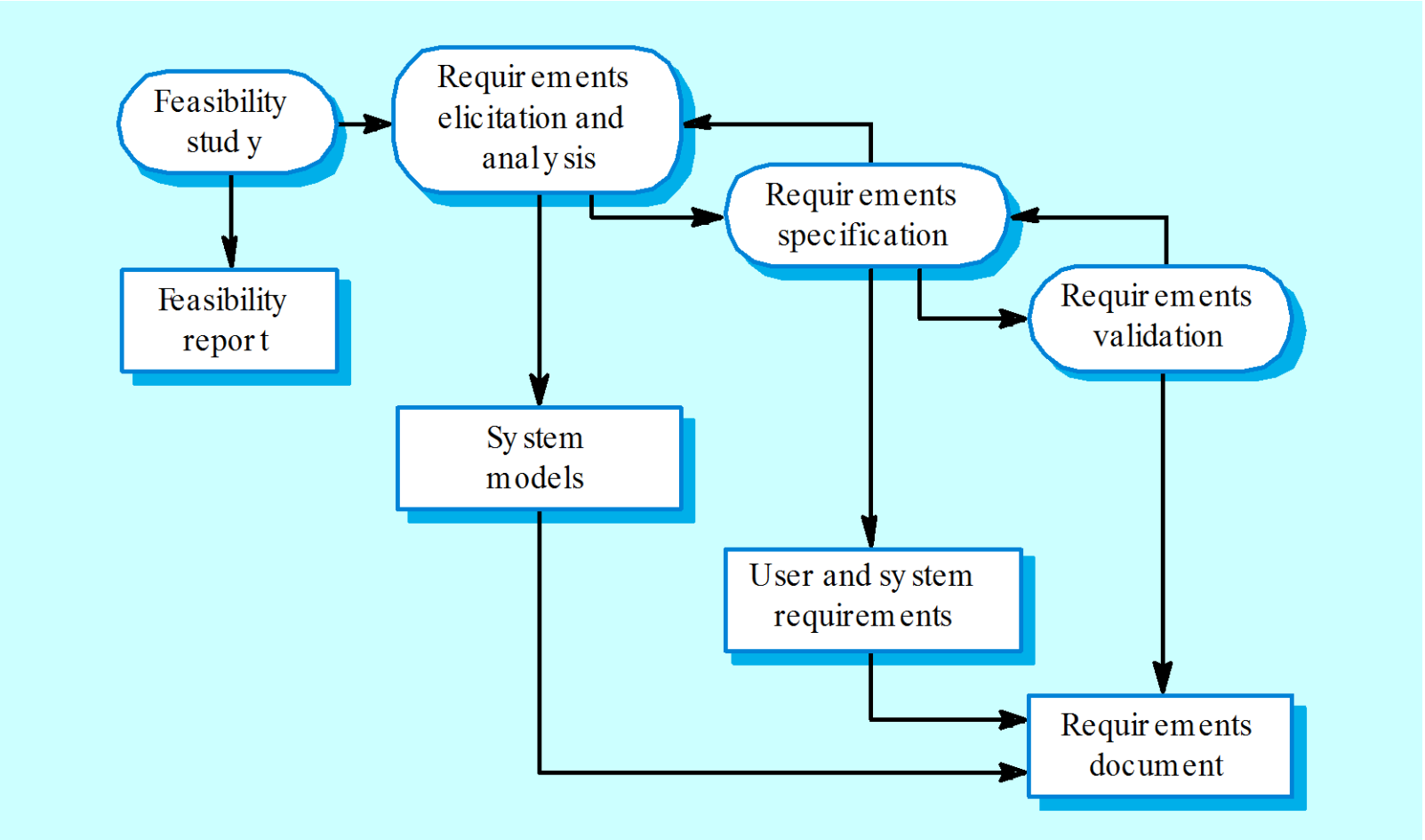
Requirements engineering

- The process of establishing the services that the customer requires from a system and the constraints under which it operates and is developed.
- The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

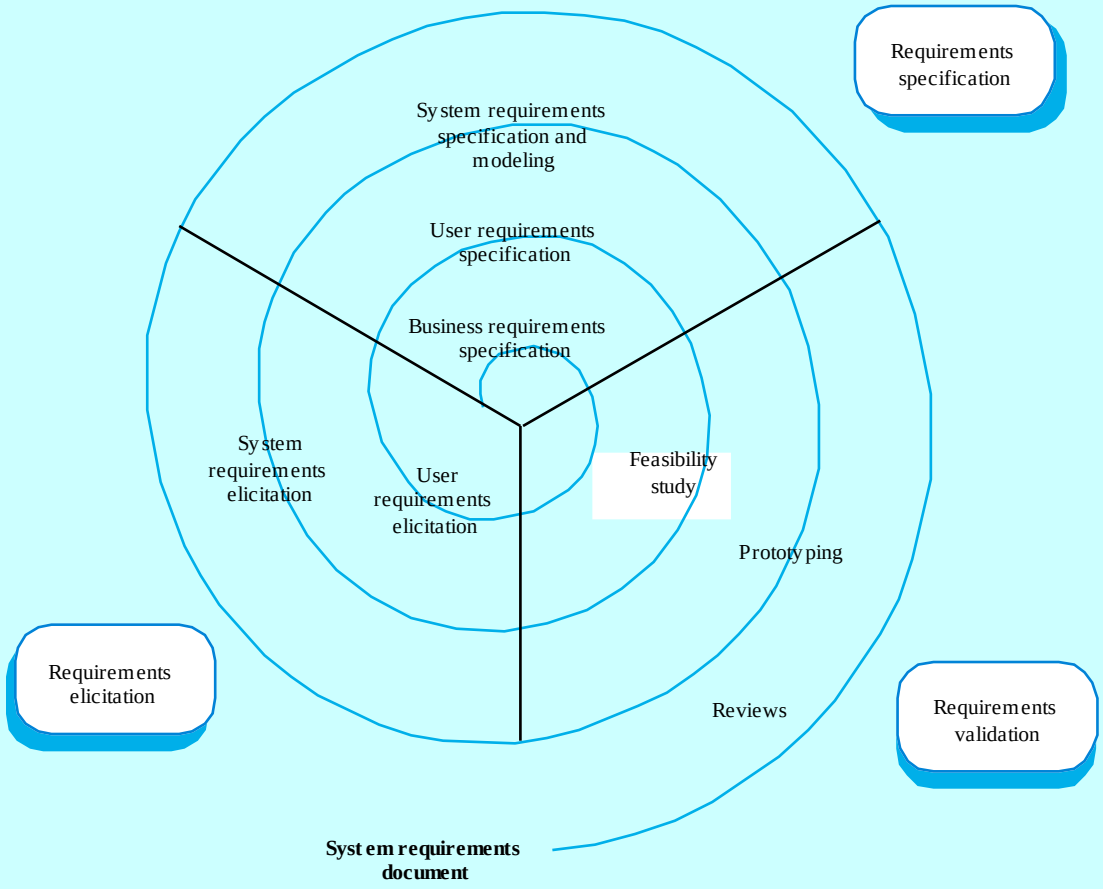
Requirements engineering processes

- The processes used for RE vary widely depending on the application domain, the people involved and the organisation developing the requirements.
- However, there are a number of generic activities common to all processes
  - Requirements elicitation;
  - Requirements analysis;
  - Requirements validation;
  - Requirements management.

The requirements engineering process



Requirements engineering



Feasibility studies

- A feasibility study decides whether or not the proposed system is worthwhile.
- A short focused study that checks
  - If the system contributes to organisational objectives;
  - If the system can be engineered using current technology and within budget;
  - If the system can be integrated with other systems that are used.

Feasibility study implementation

- Based on information assessment (what is required), information collection and report writing.
- Questions for people in the organisation
  - What if the system wasn't implemented?
  - What are current process problems?
  - How will the proposed system help?
  - What will be the integration problems?
  - Is new technology needed? What skills?
  - What facilities must be supported by the proposed system?

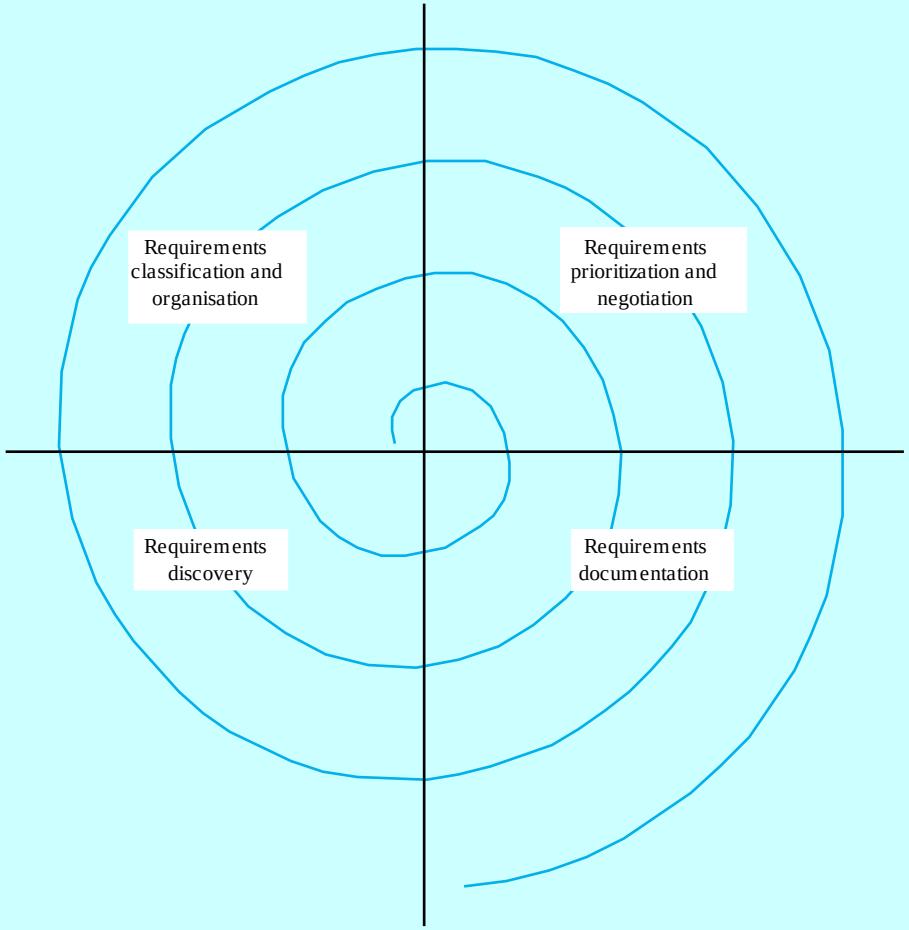
Elicitation and analysis

- A.k.a. *requirements elicitation* or *discovery*.
- *Involves technical staff working with customers to find out about the application domain, the services that the system should provide and the system's operational constraints*.
- May involve *end-users*, *managers*, *engineers involved in maintenance*, *domain experts*, *trade unions*, etc. These are called *stakeholders*.

Problems of requirements analysis

- Stakeholders
  - don’t know what they really want.
  - express requirements in their own terms.
  - may have conflicting requirements.
- Organisational and political factors may influence the system requirements.
- The requirements change during the analysis process. New stakeholders may emerge and the business environment may change.

The requirements spiral



Process activities

- Requirements discovery
  - Interacting with stakeholders to discover their requirements. Domain requirements are also discovered at this stage.
- Requirements classification and organisation
  - Groups related requirements and organises them into coherent clusters.
- Prioritisation and negotiation
  - Prioritising requirements and resolving requirements conflicts.
- Requirements documentation
  - Requirements are documented and input into the next round of the spiral.

Requirements discovery

- The process of gathering information about the proposed and existing systems and distilling the user and system requirements from this information.
- Sources of information
  - documentation,
  - system stakeholders and
  - the specifications of similar systems.

ATM stakeholders

- Bank customers
- Representatives of other banks
- Bank managers
- Counter staff
- Database administrators
- Security managers
- Marketing department
- Hardware and software maintenance engineers
- Banking regulators

Viewpoints

- Viewpoints are a way of structuring the requirements to represent the *perspectives of different stakeholders*. Stakeholders may be classified under different viewpoints.
- This *multi-perspective analysis* is *important as there is no single correct way to analyse system requirements*.

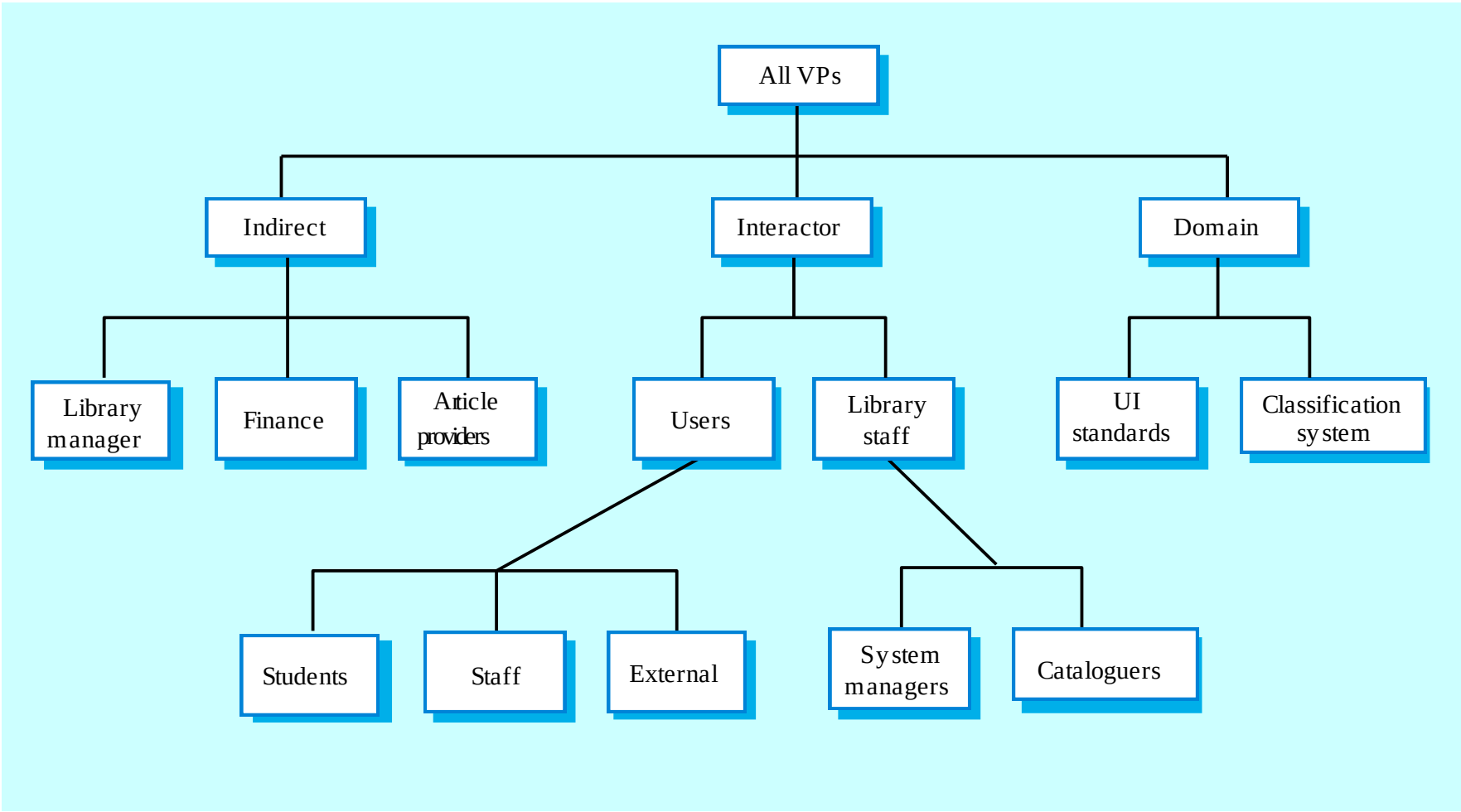
Types of viewpoint

- Interactor viewpoints
  - People or other systems that interact directly with the system. In an ATM, the customer's and the account database are interactor VPs.
- Indirect viewpoints
  - Stakeholders who do not use the system themselves but who influence the requirements. In an ATM, management and security staff are indirect viewpoints.
- Domain viewpoints
  - Domain characteristics and constraints that influence the requirements. In an ATM, an example would be standards for inter-bank communications.

Viewpoint identification

- Identify viewpoints using
  - Providers and receivers of system services;
  - Systems that interact directly with the system being specified;
  - Regulations and standards;
  - Sources of business and non-functional requirements.
  - Engineers who have to develop and maintain the system;
  - Marketing and other business viewpoints.

LIBSYS viewpoint hierarchy



Interviewing

- In formal or informal interviewing, the RE team puts questions to stakeholders about the system that they use and the system to be developed.
- There are two types of interview
  - Closed interviews where a pre-defined set of questions are answered.
  - Open interviews where there is no pre-defined agenda and a range of issues are explored with stakeholders.

Interviews in practice

- Normally a mix of closed and open-ended interviewing.
- Interviews are good for getting an overall understanding of what stakeholders do and how they might interact with the system.
- Interviews are not good for understanding domain requirements
  - Requirements engineers cannot understand specific domain terminology;
  - Some domain knowledge is so familiar that people find it hard to articulate (açıkça ifade etmek) or think that it isn’t worth articulating.

Effective interviewers

- Interviewers should be open-minded, willing to listen to stakeholders and should not have pre-conceived ideas about the requirements.
- They should prompt the interviewee with a question or a proposal and should not simply expect them to respond to a question such as ‘what do you want’.

Scenarios

- Scenarios are real-life examples of how a system can be used.
- They should include
  - A description of the starting situation;
  - A description of the normal flow of events;
  - A description of what can go wrong;
  - Information about other concurrent activities;
  - A description of the state when the scenario finishes.

LIBSYS scenario (1)

Initial assumption: The user has logged on to the LIBSYS system and has located the journal containing the copy of the article.

Normal: The user selects the article to be copied. He or she is then prompted by the system to either provide subscriber information for the journal or to indicate how they will pay for the article. Alternative payment methods are by credit card or by quoting an organisational account number.

The user is then asked to fill in a copyright form that maintains details of the transaction and they then submit this to the LIBSYS system.

The copyright form is checked and, if OK, the PDF version of the article is downloaded to the LIBSYS working area on the user's computer and the user is informed that it is available. The user is asked to select a printer and a copy of the article is printed. If the article has been flagged as 'print-only' it is deleted from the user's system once the user has confirmed that printing is complete.

LIBSYS scenario (2)

What can go wrong: The user may fail to fill in the copyright form correctly. In this case, the form should be re-presented to the user for correction. If the resubmitted form is still incorrect then the user's request for the article is rejected.

The payment may be rejected by the system. The user's request for the article is rejected.

The article download may fail. Retry until successful or the user terminates the session.

It may not be possible to print the article. If the article is not flagged as 'print-only' then it is held in the LIBSYS workspace. Otherwise, the article is deleted and the user's account credited with the cost of the article.

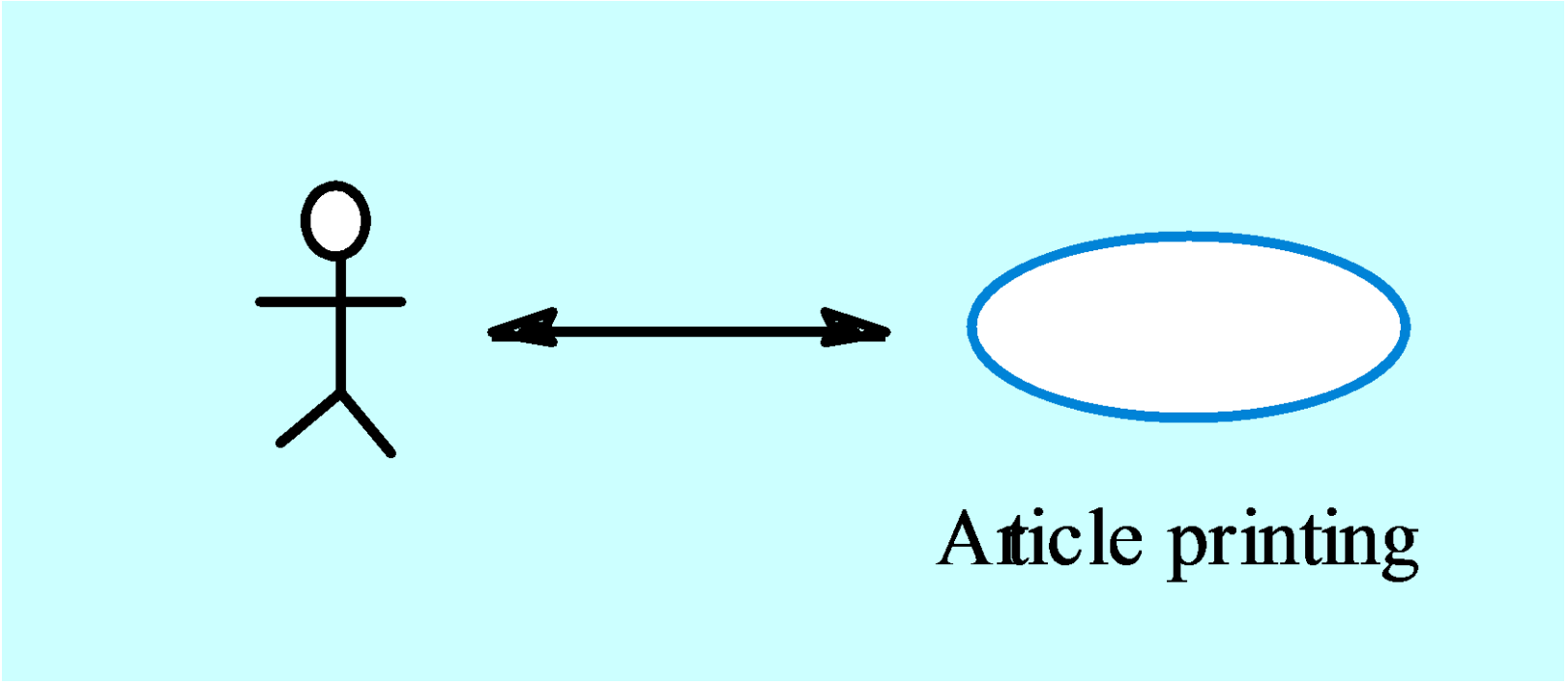
Other activities: Simultaneous downloads of other articles.

System state on completion: User is logged on. The downloaded article has been deleted from LIBSYS workspace if it has been flagged as print-only.

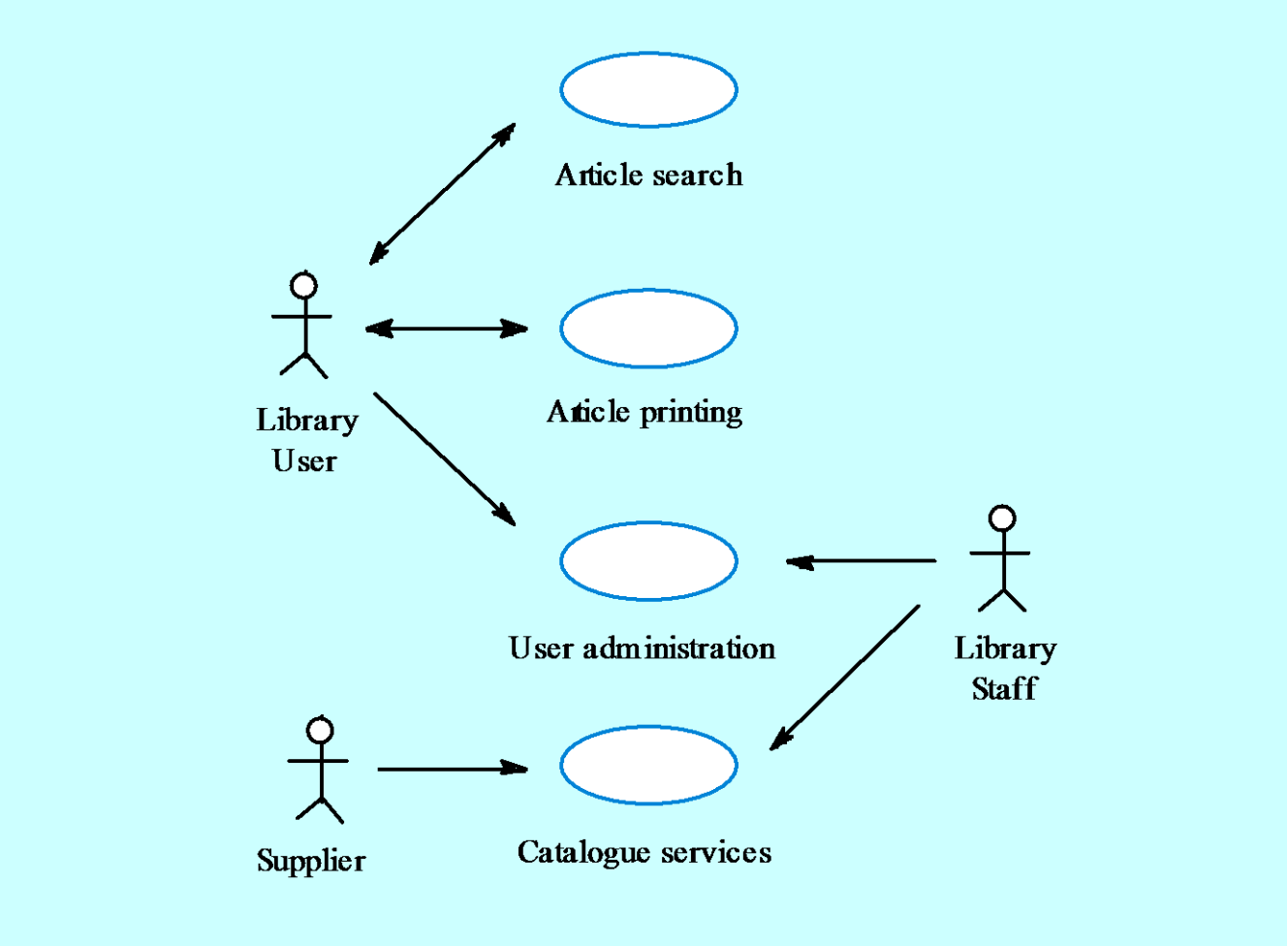
Use cases

- Use-cases are a scenario based technique in the UML which identify the actors in an interaction and which describe the interaction itself.
- A set of use cases should describe all possible interactions with the system.
- Sequence diagrams may be used to add detail to use-cases by showing the sequence of event processing in the system.

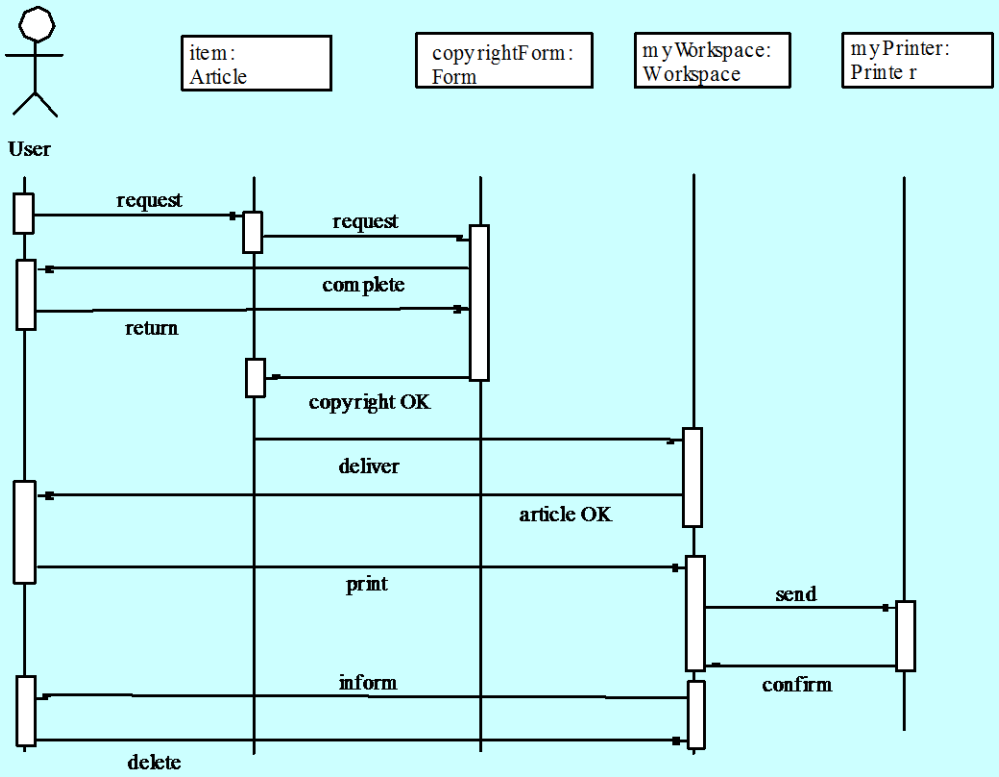
Article printing use-case



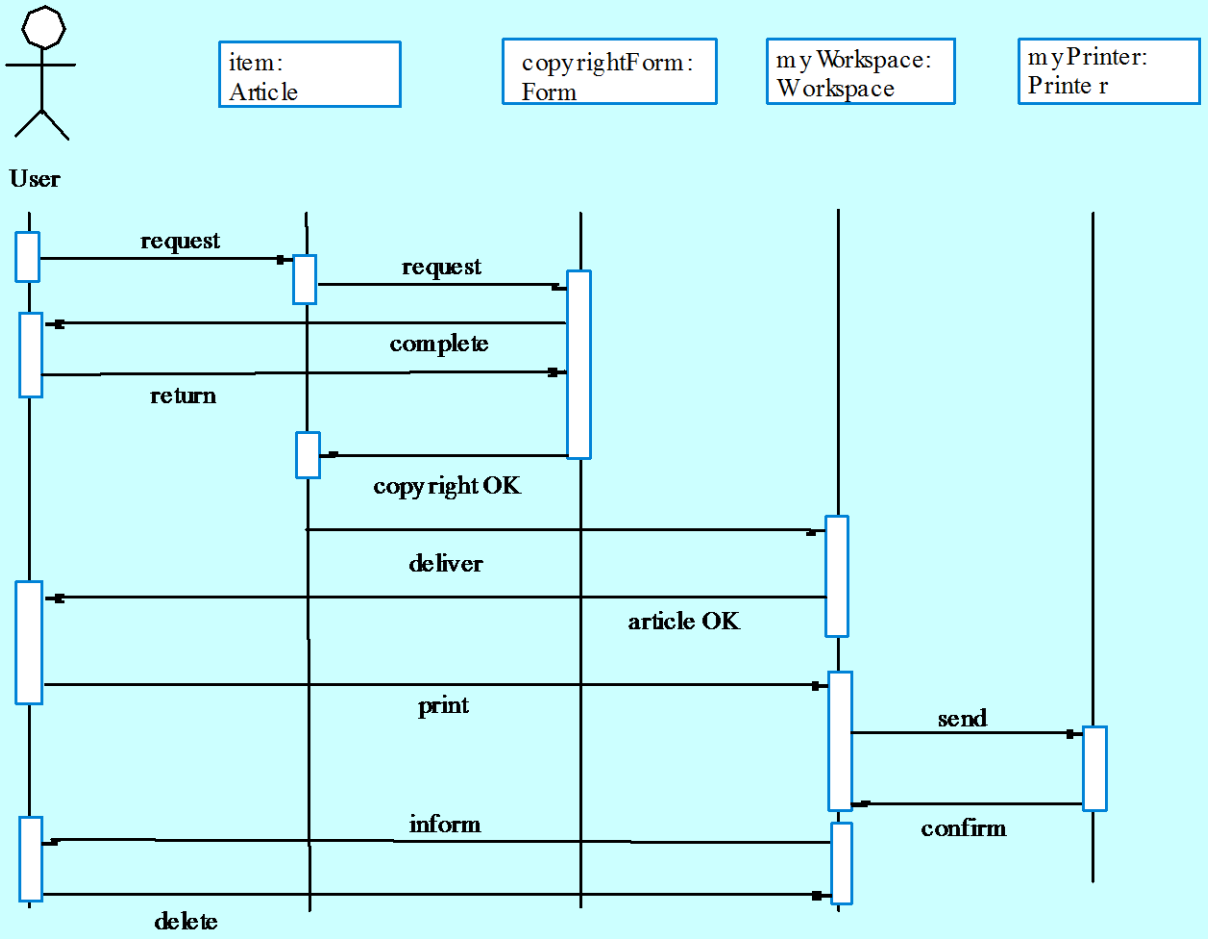
LIBSYS use cases



Article printing



Print article sequence



Social and organisational factors

- Software systems are used in a social and organisational context. This can influence or even dominate the system requirements.
- Social and organisational factors are not a single viewpoint but are influences on all viewpoints.
- Good analysts must be sensitive to these factors but currently no systematic way to tackle their analysis.

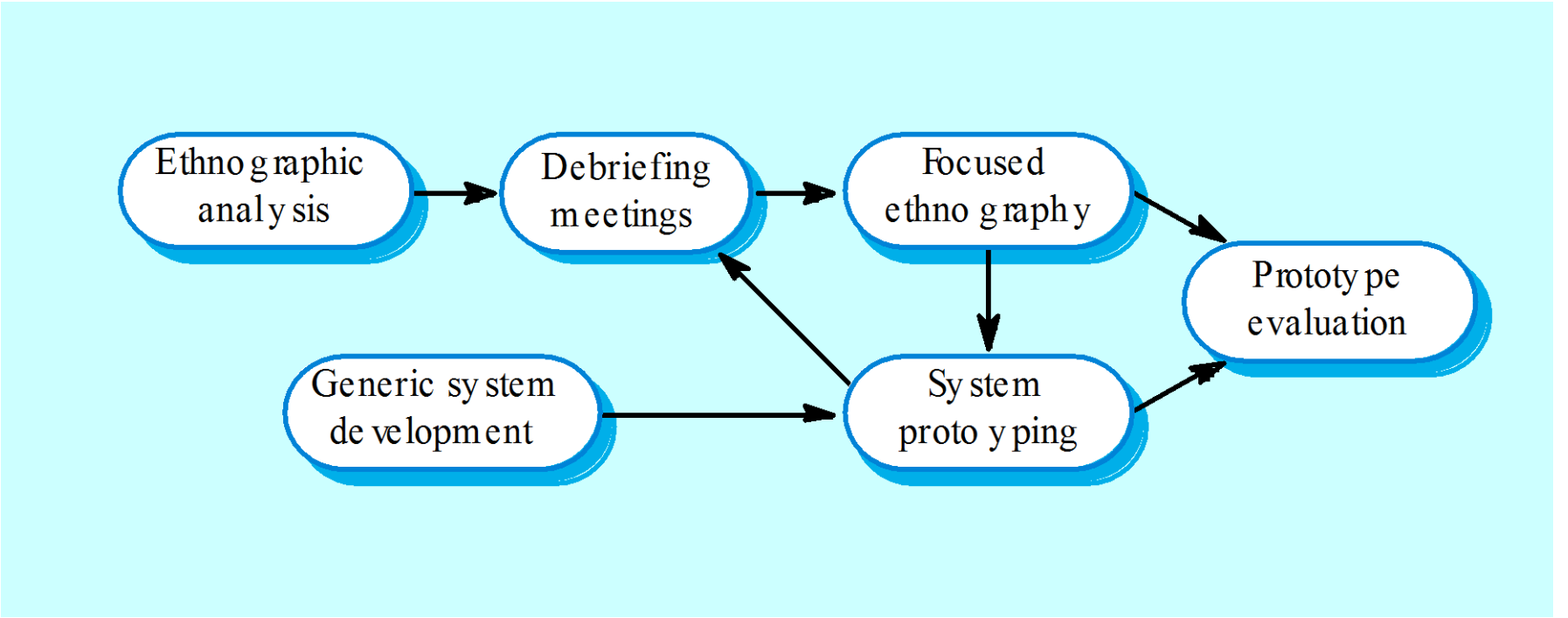
Ethnography

- A *social scientist spends a considerable time observing and analysing how people actually work*.
- *People do not have to explain or articulate their work*.
- Social and organisational factors of importance may be observed.
- Ethnographic studies have shown that work is usually richer and more complex than suggested by simple system models.

Focused ethnography

- Developed in a project studying the air traffic control process.
- Combines ethnography with prototyping.
- Prototype development results in unanswered questions focusing on ethnographic analysis.
- The problem with ethnography is that it studies existing practices which may have some historical basis which is no longer relevant.

Ethnography and prototyping



Scope of ethnography

- Requirements that are derived from the way that *people actually work rather than the way in which process definitions suggest that they should work*.
- Requirements that are derived from cooperation and awareness of other people's activities.

Requirements validation

- Concerned with demonstrating that the requirements define the system that the customer really wants.
- Requirements error costs are high so validation is very important
  - Fixing a requirements error after delivery may cost up to 100 times the cost of fixing an implementation error.

Requirements checking

- Validity. Does the system provide the functions which best support the customer's needs?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology
- Verifiability. Can the requirements be checked?

Requirements validation techniques

- Requirements reviews
  - Systematic manual analysis of the requirements.
- Prototyping
  - Using an executable model of the system to check requirements. Covered in Chapter 17.
- Test-case generation
  - Developing tests for requirements to check testability.

Requirements reviews

- Regular reviews should be held while the requirements definition is being formulated.
- Both client and contractor staff should be involved in reviews.
- Reviews may be formal (with completed documents) or informal. Good communications between developers, customers and users can resolve problems at an early stage.

Review checks

- Verifiability. Is the requirement realistically testable?
- Comprehensibility. Is the requirement properly understood?
- Traceability. Is the origin of the requirement clearly stated?
- Adaptability. Can the requirement be changed without a large impact on other requirements?

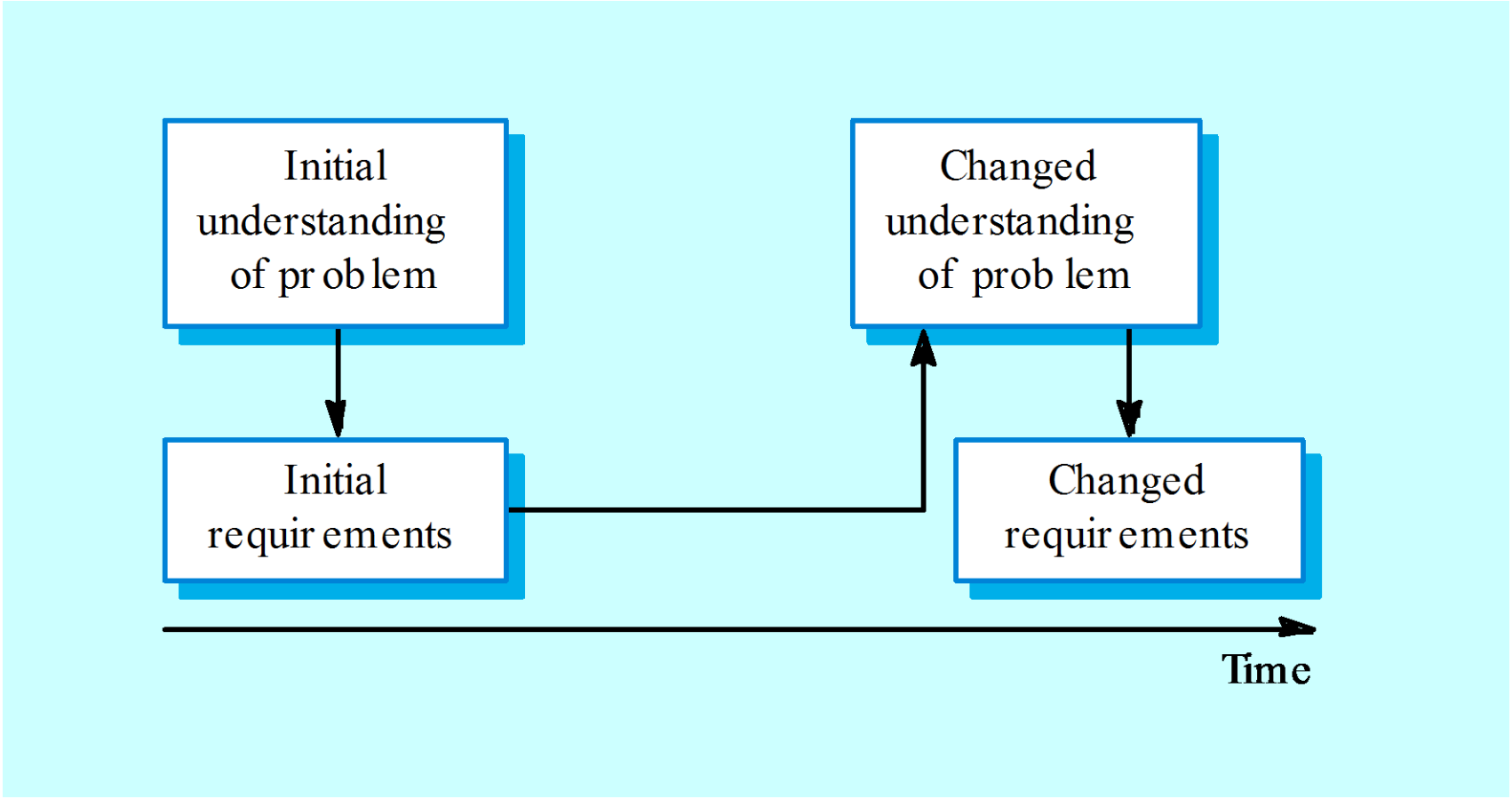
Requirements management

- Requirements management is the process of managing changing requirements during the requirements engineering process and system development.
- Requirements are inevitably incomplete and inconsistent
  - New requirements emerge during the process as business needs change and a better understanding of the system is developed;
  - Different viewpoints have different requirements and these are often contradictory.

Requirements change

- The priority of requirements from different viewpoints changes during the development process.
- System customers may specify requirements from a business perspective that conflict with end-user requirements.
- The business and technical environment of the system changes during its development.

Requirements evolution



Enduring and volatile requirements

- Enduring requirements. Stable requirements derived from the core activity of the customer organisation. E.g. a hospital will always have doctors, nurses, etc. May be derived from domain models
- Volatile requirements. Requirements which change during development or when the system is in use. In a hospital, requirements derived from health-care policy

Requirements classification

Requirement Type	Description
Mutable requirements	Requirements that change because of changes to the environment in which the organisation is operating. For example, in hospital systems, the funding of patient care may change and thus require different treatment information to be collected.
Emergent requirements	Requirements that emerge as the customer's understanding of the system develops during the system development. The design process may reveal new emergent requirements.
Consequential requirements	Requirements that result from the introduction of the computer system. Introducing the computer system may change the organisations processes and open up new ways of working which generate new system requirements
Compatibility requirements	Requirements that depend on the particular systems or business processes within an organisation. As these change, the compatibility requirements on the commissioned or delivered system may also have to evolve.

Requirements management planning

- During the requirements engineering process, you have to plan:
  - Requirements identification
    - How requirements are individually identified;
  - A change management process
    - The process followed when analysing a requirements change;
  - Traceability policies
    - The amount of information about requirements relationships that is maintained;
  - CASE tool support
    - The tool support required to help manage requirements change;

Traceability

- Traceability is concerned with the relationships between requirements, their sources and the system design
- Source traceability
  - Links from requirements to stakeholders who proposed these requirements;
- Requirements traceability
  - Links between dependent requirements;
- Design traceability
  - Links from the requirements to the design;

A traceability matrix

Req. id	1.1	1.2	1.3	2.1	2.2	2.3	3.1	3.2
1.1		D	R					
1.2			D			D		D
1.3	R			R				
2.1			R		D			D
2.2								D
2.3		R		D				
3.1								R
3.2							R	

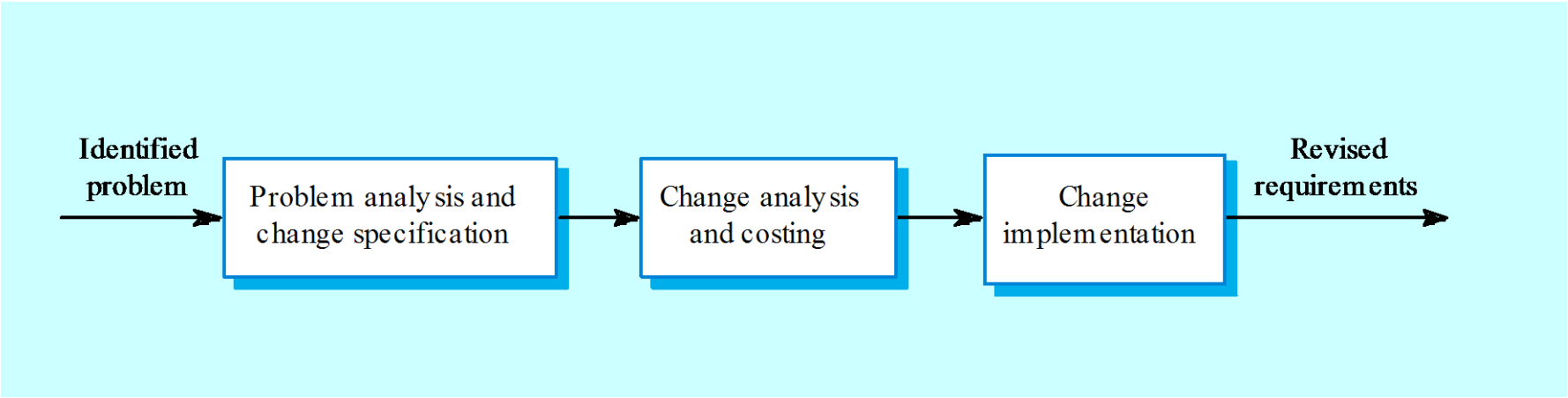
CASE tool support

- Requirements storage
  - Requirements should be managed in a secure, managed data store.
- Change management
  - The process of change management is a workflow process whose stages can be defined and information flow between these stages partially automated.
- Traceability management
  - Automated retrieval of the links between requirements.

Requirements change management

- Should apply to all proposed changes to the requirements.
- Principal stages
  - Problem analysis. Discuss requirements problem and propose change;
  - Change analysis and costing. Assess effects of change on other requirements;
  - Change implementation. Modify requirements document and other documents to reflect change.

Change management



Key points

- The requirements engineering process includes a feasibility study, requirements elicitation and analysis, requirements specification and requirements management.
- Requirements elicitation and analysis is iterative involving domain understanding, requirements collection, classification, structuring, prioritisation and validation.
- Systems have multiple stakeholders with different requirements.

Key points

- Social and organisation factors influence system requirements.
- Requirements validation is concerned with checks for validity, consistency, completeness, realism and verifiability.
- Business changes inevitably lead to changing requirements.
- Requirements management includes planning and change management.

Quiz 2 – Part 1 - Week 3

1) Who prepares the user requirements?

2) Who prepares the system requirements?

Quiz 2 – Part 2 - Week 3

1) _____ requirements are meant for a basis for a bid while _____ requirements are meant for a basis for a contract.
2) Mostly technical people are the readers of the _____ requirements.
3) _____ requirements are defined using natural languages, tables and diagrams while _____ requirements are expressed by alternatives to the natural language.

Quiz 2 – Part 3 - Week 3

1) What are the problems with natural language (NL) specification?

2) What are the alternatives to NL specification?

3) Who prepares the user requirements?

4) Who prepares the system requirements?